Naya Prayog Academy — Python DS & AI

Real-World Practice Questions

(Basics Day 1–22 + Analytics Day 1–Day 6 Seaborn)

Instructions for Students:

These problems are designed like real tasks you will face as a Data Analyst, Data Scientist, or AI/ML Engineer.

Each problem gives you:

- A scenario
- A dataset to create (or is provided)
- Tasks to complete

Do NOT look at solutions first. Try on your own. Think like a professional.

MODULE 1 — PYTHON BASICS

PROJECT 1: Student Grade Manager

Role: You are a junior developer at a school management company.

Scenario: Build a console program that manages student grades.

Tasks:

1. Ask the user to enter a student's name, marks in 5 subjects (Math, Science, English, Hindi, Computer).
2. Calculate the total marks and percentage.
3. Assign a grade using the following rules:

- 90% and above → Grade A
- 75–89% → Grade B
- 60–74% → Grade C
- 40–59% → Grade D
- Below 40% → Grade F (Fail)

4. Print a formatted report card like this:

1. Student Report Card
2.
3. Name   : Rahul Kumar
4. Total   : 420 / 500
5. Percentage: 84.00%
6. Grade   : B

5. Handle the case where the user enters an invalid mark (not a number) using exception handling.

Concepts used: variables, input(), int/float conversion, f-strings, if-elif-else, try-except

PROJECT 2: E-Commerce Cart System

Role: Junior Python Developer at an online shopping startup.

Scenario: Build a simple shopping cart using Python data structures.

Tasks:

1. Create a dictionary called `product_catalog` with at least 6 products and their prices.
2. Create an empty list called `cart`.
3. Write a menu-driven program (using a while loop) that lets the user:

- View all products with prices
- Add a product to cart by name
- Remove a product from cart
- View current cart and total bill
- Exit the program

4. If the user tries to add a product that doesn't exist, print:
7. Product not found in catalog.

5. Show the total bill with 18% GST added.

Concepts used: dictionary, list, while loop, if-elif, break, in operator, f-strings

PROJECT 3: Zomato Delivery Time Estimator

Role: Data Operations Analyst at a food delivery company.

Scenario: Build a program that estimates delivery time based on distance and traffic.

Tasks:

1. Ask the user:
- Distance in km (float)
- Traffic level: 1 = Low, 2 = Medium, 3 = High
2. Use these rules to calculate delivery time:
- Low traffic: speed = 40 km/h
- Medium traffic: speed = 25 km/h
- High traffic: speed = 15 km/h
3. Calculate time in minutes:
8. time = (distance / speed) * 60

4. Add a preparation time of 15 minutes.
5. Print result:
9. Estimated delivery time: 42 minutes

6. If the user enters an invalid traffic level (not 1, 2, or 3), show an error using exception handling.
7. Bonus: Use match-case instead of if-elif for traffic level.

Concepts used: input, float conversion, if-elif or match-case, arithmetic, try-except

PROJECT 4: Number Pattern Analyzer

Role: Backend Developer — Data Validation Team

Scenario: Your company needs a tool that analyzes a list of numbers for patterns.

Tasks:

1. Create a list of 20 numbers entered by the user (use a loop to collect input).
2. Using list comprehension, create:

- `even_numbers` — all even numbers from the list
- `odd_numbers` — all odd numbers
- `squared_values` — square of every number
- `above_average` — numbers greater than the average of the list

3. Print all results in a clean format.
4. Find and print the maximum, minimum, and average of the list WITHOUT using max() and min().
5. Check if any number in the list is prime. Print all prime numbers found.

Concepts used: list, for loop, list comprehension, while loop, if conditions, user input, exception handling

PROJECT 5: Bank Account System

Role: Junior Developer at a FinTech Startup

Scenario: Create a simple bank account simulation.

Tasks:

1. Write the following functions:

- `create_account(name, initial_balance)`
- `deposit(account, amount)`
- `withdraw(account, amount)`
- `check_balance(account)`
- `transaction_history(account)`

2. Each transaction (deposit/withdraw) should be stored in a list inside the account dictionary.
3. Handle invalid inputs (negative amounts, non-numeric) using try-except.
4. Run a demo:

- Create an account
- Do 3 deposits
- Do 2 withdrawals
- Print history